

JPA (*Java Persistence API*) - Partie 2

Université de Nice - Sophia Antipolis
Version 1.4 – 13/7/08
Richard Grin

Plan de la partie 2 (1)

- ❑ Requêtes et langage d'interrogation
- ❑ Modifications en volume
- ❑ Exceptions
- ❑ Transaction
- ❑ Concurrency
- ❑ Entités détachées
- ❑ Configuration d'une unité de persistance
- ❑ Fichiers de configuration XML

R. Grin

JPA

page 2

Plan de la partie 2 (2)

- ❑ JPA et DAOs
- ❑ Optimisation
- ❑ Callbacks

R. Grin

JPA

page 3

Requêtes - JPQL

R. Grin

JPA

page 4

- ❑ Cette section concerne les entités (et valeurs) retrouvées en interrogeant la base de données

R. Grin

JPA

page 5

Rappel important

- ❑ Toutes les entités retrouvées par **find**, **getReference** ou un query sont automatiquement gérées par le gestionnaire d'entités
- ❑ Les modifications apportées à ces entités sont donc enregistrées au prochain commit (mais les nouveaux objets associés à l'entité retrouvée ne sont pas automatiquement persistants même s'il y a une cascade sur **persist** ; ça ne marche que pour un appel explicite de la méthode **persist**)

R. Grin

JPA

page 6

Chercher par identité

- ❑ **find** et **getReference** (de **EntityManager**) permettent de retrouver une entité en donnant son identificateur dans la BD
- ❑ **<T> T find(Class<T> classeEntité, Object cléPrimaire)**
- ❑ **<T> T get(Class<T> classeEntité, Object cléPrimaire)**
- ❑ Le résultat sera **null** si aucune entité n'a cet identificateur dans la base de donnée

Exemple

```
Departement dept =  
    em.find(Departement.class, 10);
```

getReference

- ❑ **getReference** renvoie une référence vers une entité, sans que cette entité ne soit nécessairement initialisée
- ❑ Le plus souvent il vaut mieux utiliser **find**

getReference

- ❑ **getReference** peut être (rarement) utilisée pour améliorer les performances quand une entité non initialisée peut être suffisante, sans que l'entité entière soit retrouvée dans la base de données
- ❑ Par exemple, pour indiquer une association dont le but est unique (OneToOne ou ManyToOne) :

```
Departement dept =  
    em.getReference(Departement.class, 10);  
Employe emp.setDepartement(dept);
```

Étapes pour récupérer des données

- ❑ Il est possible de rechercher des données sur des critères plus complexes que la simple identité
- ❑ Les étapes sont alors les suivantes :
 1. Décrire ce qui est recherché (langage JPQL)
 2. Créer une instance de type **Query**
 3. Initialiser la requête (paramètres, pagination)
 4. Lancer l'exécution de la requête

Langage JPQL

- ❑ Le langage JPQL (*Java Persistence Query Language*) permet de décrire ce que l'application recherche
- ❑ Il ressemble beaucoup à SQL

Requêtes sur les entités « objet »

- ❑ Les requêtes JPQL travaillent avec le modèle objet et pas avec le modèle relationnel
- ❑ Les identificateurs désignent les entités et leurs propriétés et pas les tables et leurs colonnes
- ❑ Les seules classes qui peuvent être explicitement désignées dans une requête (clause **from**) sont les entités

Désigner les entités

- ❑ Les entités sont désignées par leur nom
- ❑ Le nom d'une entité est donné par l'attribut **name** de **@Entity** ; par défaut c'est le nom terminal (sans le nom du paquetage) de la classe

Alias

- ❑ Le texte des requêtes utilise beaucoup les alias de classe
- ❑ Les attributs des classes doivent être préfixés par les alias
- ❑ Une erreur fréquente du débutant est d'oublier les alias en préfixe

Exemples de requêtes JPQL

- ❑ `select e from Employe as e`
- ❑ `select e.nom, e.salaire from Employe e`
- ❑ `select e from Employe e
where e.departement.nom = 'Direction'`
- ❑ `select d.nom, avg(e.salaire)
from Departement d join d.employes e
group by d.nom
having count(d.nom) > 5`

Type du résultat (1)

- ❑ L'expression de la clause select peut être
 - une (ou plusieurs) expression « entité », par exemple un employé (**e** par exemple)
 - une (ou plusieurs) expression « valeur », par exemple le nom et le salaire d'un employé (**e.nom** par exemple)
- ❑ Si la requête renvoie des entités, elles sont automatiquement gérées par le GE (toute modification sera répercutée dans la base)

Type du résultat (2)

- ❑ L'expression ne peut être une collection (**d.employes** par exemple), bien que TopLink le permette !

Obtenir le résultat de la requête

- ❑ Pour le cas où une seule valeur ou entité est renvoyée, le plus simple est d'utiliser la méthode **getSingleResult()** de l'interface **Query** ; elle renvoie un **Object**
- ❑ La méthode lance des exceptions s'il n'y a pas exactement une entité qui correspond à la requête :
 - **EntityNotFoundException**
 - **NonUniqueResultException**

Obtenir le résultat de la requête

- ❑ Si plusieurs valeurs ou entités peuvent être renvoyées, il faut utiliser la méthode **getResultList()** de l'interface **Query**
- ❑ Elle renvoie une liste « raw » (pas générique) des résultats, instance de **java.util.List**, éventuellement vide si le résultat est vide
- ❑ Un message d'avertissement est affiché durant la compilation si le résultat est rangé dans une liste générique (**List<Employe>** par exemple)

Type d'un élément du résultat

- ❑ Le type d'un élément de la liste (ou de l'unique valeur ou entité renvoyée) est
 - **Object** si la clause select ne comporte qu'une seule expression
 - **Object[]** si elle comporte plusieurs expressions

Exemple 1

```
String s = "select e from Employe as e";
Query query = em.createQuery(s);
List<Employe> listeEmployes =
    (List<Employe>)query.getResultList();
```

Exemple 2

```
texte = "select e.nom, e.salaire "
        + " from Employe as e";
query = em.createQuery(texte);
List<Object[]> liste =
    (List<Object[]>)query.getResultList();
for (Object[] info : liste) {
    System.out.println(info[0] + " gagne "
        + info[1]);
}
```

Interface Query

- ❑ Représente une requête
- ❑ Une instance de **Query** (d'une classe implémentant **Query**) est obtenue par les méthodes **createQuery**, **createNativeQuery** ou **createNamedQuery** de l'interface **EntityManager**

Méthodes de Query (1)

- ❑ `List getResultList()`
- ❑ `Object getSingleResult()`
- ❑ `int executeUpdate()`
- ❑ `Query setMaxResults(int nbResultats)`
- ❑ `Query setFirstResult(int positionDepart)`
- ❑ `Query setFlushMode(FlushModeType modeFlush)`

Méthodes de Query (2)

- ❑ `Query setParameter(String nom, Object valeur)`
- ❑ `Query setParameter(String nom, Date valeur, TemporalType typeTemporel)`
- ❑ `Query setParameter(String nom, Calendar valeur, TemporalType typeTemporel)`

Types temporels

- ❑ On a vu que les 2 types java temporels du paquetage `java.util` (`Date` et `Calendar`) nécessitent une annotation `@Temporal`
- ❑ Ils nécessitent aussi un paramètre supplémentaire pour la méthode `setParameter`

Exemple

- ❑ `@Temporal(TemporalType.DATE)`
`private Calendar dateEmb;`
- ❑ `em.createQuery("select e from employe e" + " where e.dateEmb between ?1 and ?2")`
`.setParameter(1, debut, TemporalType.DATE)`
`.setParameter(2, fin, TemporalType.DATE)`
`.getResultList();`

Types de requête

- ❑ Requêtes dynamiques dont le texte est donnée en paramètre de `createQuery`
- ❑ Requêtes natives (appelées aussi requêtes SQL) particulières à un SGBD ou trop complexes pour JPA ; requête SQL (pas JPQL) avec tables et colonnes (pas classes et attributs)
- ❑ Requêtes nommées dont le texte est donnée dans une annotation de l'entité concernée et dont le nom est passé en paramètre de `createNamedQuery` ; une requête nommée peut être dynamique ou native

Paramètres des requêtes

- ❑ Un paramètre peut être désigné par son numéro (`?n`) ou par son nom (`:nom`)
- ❑ Les valeurs des paramètres sont données par les méthodes `setParameter`
- ❑ Les paramètres sont numérotés à partir de 1
- ❑ Un paramètre peut être utilisé plus d'une fois dans une requête
- ❑ L'usage des paramètres nommés est recommandé (plus lisible)

Requête nommée (1)

- ❑ Seules les entités peuvent contenir des définitions de requêtes nommées
- ❑ Une requête nommée peut être mise dans n'importe quelle entité, mais on choisira le plus souvent l'entité qui correspond à ce qui est renvoyé par la requête
- ❑ Le nom de la requête nommée doit être unique parmi toutes les entités de l'unité de persistance ; on pourra, par exemple, préfixer le nom par le nom de l'entité : `Employe.findAll`

Requête nommée (2)

- ❑ Les requêtes nommées peuvent être analysées et précompilées par le fournisseur de persistance au démarrage de l'application, ce qui peut améliorer les performances

Exemple de requête nommée

```
@Entity
@NamedQuery (
    name = "findNomsEmployes",
    query = "select e.nom from Employe as e
            where upper(e.departement.nom) = :nomDept"
)
public class Employe extends Personne {
    ...
}
```

```
Query q =
    em.createNamedQuery("findNomsEmployes");
```

Exemples

- ❑ `Query query = em.createQuery("select e from Employe as e " + "where e.nom = ?1"); query.setParameter(1, "Dupond");`
- ❑ `Query query = em.createQuery("select e from Employe as e " + "where e.nom = :nom"); query.setParameter("nom", "Dupond");`

Plusieurs requêtes nommées

- ❑ Si une classe a plusieurs requêtes nommées, il faut les regrouper dans une annotation

@NamedQueries :

```
@NamedQueries({
    @NamedQuery(...),
    @NamedQuery(...),
    ...
})
```

Mapping pour requêtes natives (1)

- ❑ Il est nécessaire de donner des informations sur les colonnes renvoyées par la requête SQL afin que le gestionnaire d'entités sache si elle correspondent à des entités
- ❑ Dans le cas où toutes les colonnes correspondent aux attributs d'une entité ces informations de mapping sont indiquées en donnant la classe de l'entité en paramètre de **createNativeQuery**

Mapping pour requêtes natives (2)

- ❑ Dans les autres cas, une annotation **@SqlResultSetMapping** est nécessaire
- ❑ Elle doit être ajoutée à une entité quelconque
- ❑ Cette annotation a un nom qui doit être passé en paramètre à la méthode **createNativeQuery**
- ❑ Consultez une documentation sur JPA pour en savoir plus

Requêtes natives ou JDBC ?

- ❑ Si une entité est renvoyée par la requête, il vaut mieux utiliser une requête native car le gestionnaire d'entités gèrera l'entité comme les autres entités
- ❑ Si des valeurs scalaires sont renvoyées, le choix est moins clair ; utiliser une requête nommée permettra de rester dans JPA
- ❑ Une requête trop complexe nécessitera JDBC ; de même si le texte de la requête est généré lors de l'exécution

Mode de flush

- ❑ Normalement (mode **FlushMode.AUTO**) un flush des entités concernées par une requête est effectué avant la requête pour que le résultat tienne compte des modifications effectuées en mémoire sur ces entités
- ❑ Pour une requête il est possible d'éviter ce flush avec la méthode **setFlushMode** :
`query.setFlushMode(FlushMode.COMMIT);`
Dans ce mode, seul un commit provoquera un flush

- ❑ Les transparents suivants étudient en détails le langage JPQL

Clauses d'un select

- ❑ **select** : type des objets ou valeurs renvoyées
- ❑ **from** : où les données sont récupérées
- ❑ **where** : sélectionne les données
- ❑ **group by** : regroupe des données
- ❑ **having** : sélectionne les groupes (ne peut exister sans clause group by)
- ❑ **order by** : ordonne les données

- ❑ Les mots-clés **select**, **from**, **distinct**, **join**,... sont insensibles à la casse

Polymorphisme dans les requêtes

- ❑ Toutes les requêtes sont polymorphes : un nom de classe dans la clause **from** désigne cette classe et toutes les sous-classes

- ❑ Exemple :

```
select count(a) from Article as a
```

compte le nombre d'instances de la classe **Article** et de tous les sous-classes de **Article**

Expression de chemin

- ❑ Les requêtes peuvent contenir des expressions de chemin pour naviguer entre les entités en suivant les associations déclarées dans le modèle objet (les annotations **@OneToOne**, **@OneToMany**, ...)
- ❑ La notation « pointée » est utilisée

Règle pour les expressions de chemin

- ❑ Une navigation peut être chaînée à une navigation précédente à la condition que la navigation précédente ne donne qu'une seule entité (**OneToOne** ou **ManyToOne**)
- ❑ Dans le cas où une navigation aboutit à plusieurs entités, il est possible d'utiliser la clause **join** étudiée plus loin pour obtenir ces entités

Exemples

- ❑ Si **e** est un alias pour **Employe**,
 - « **e.departement** » désigne le département d'un employé
 - « **e.projets** » désigne la collection de projets auxquels participe un employé
- ❑

```
select e.nom  
from Employe as e  
where e.departement.nom = 'Qualité'
```
- ❑ **e.projets.nom** n'est pas autorisé car **e.projets** est une collection (voir clause **join**)

distinct

- ❑ Dans une clause **select**, indique que les valeurs dupliquées sont éliminées (la requête ne garde qu'une seule des valeurs égales)

- ❑ Exemple :

```
select distinct e.departement  
from Employe e
```

new

- ❑ Il est possible de renvoyer des instances d'une classe dont le constructeur prend en paramètre des informations récupérées dans la base de données
- ❑ La classe doit être désignée par son nom complet (avec le nom du package)
- ❑ Exemple :

```
select new p1.p2.Classe(e.nom, e.salaire)  
from Employe e
```

Clauses where et having

- Ces clauses peuvent comporter les mots-clés suivants :

- [NOT] LIKE, [NOT] BETWEEN, [NOT] IN
- AND, OR, NOT
- [NOT] EXISTS
- ALL, SOME/ANY
- IS [NOT] NULL
- IS [NOT] EMPTY, [NOT] MEMBER OF (pour les collections)

Exemple

```
select d.nom, avg(e.salaire)
from Departement d join d.employees e
group by d.nom
having count(d.nom) > 3
```

having

- Restriction : la condition doit porter sur l'expression de regroupement ou sur une fonction de regroupement portant sur l'expression de regroupement
- Par exemple, la requête suivante provoque une exception :

```
select d.nom, avg(e.salaire)
from Departement d join d.employees e
group by d.nom
having avg(e.salaire) > 1000
```

Sous-requête (1)

- Les clauses **where** et **having** peuvent contenir des sous-requêtes

- Exemple :

```
select e from Employe e
where e.salaire >= (
    select e2.salaire from Employe e2
    where e2.departement = 10)
```

Sous-requête (2)

- {ALL | ANY | SOME} (sous-requête) fonctionne comme dans SQL

- Exemple :

```
select e from Employe e
where e.salaire >= ALL (
    select e2.salaire from Employe e2
    where e2.departement =
        e.departement)
```

Sous-requête synchronisée

- Une sous-requête peut être synchronisée avec une requête englobante

- Exemple :

```
select e from Employe e
where e.salaire >= ALL (
    select e2.salaire from Employe e2
    where e2.departement =
        e.departement)
```

Sous-requête - **exists**

- ❑ **[not]exists** fonctionne comme avec SQL
- ❑ Exemple :

```
select emp from Employe e
where exists (
  select ee from Employe ee
  where ee = e.epouse)
```

Exemple

```
select e.nom,
       e.departement.nom,
       e.superieur.departement.nom
from Employe e
```

Contre-exemple

- ❑ « **d.employes.nom** » est interdit car **d.employes** est une collection
- ❑ Pour avoir les noms des employés d'un département, il faut utiliser une jointure

Jointure

- ❑ Une jointure permet de combiner plusieurs entités dans un select
- ❑ Rappel : il est possible d'utiliser plusieurs entités dans une requête grâce à la navigation
- ❑ Une jointure est le plus souvent utilisée pour résoudre les cas (interdit par JPA) où
 - l'expression du select serait une collection
 - la navigation partirait d'une collection

Types de jointures

- ❑ Il existe plusieurs types de jointures :
 - jointure interne (jointure standard **join**)
 - jointure externe (**outer join**)
 - jointure avec récupération de données en mémoire (**join fetch**)
 - jointure « à la SQL » dans un where pour joindre suivant des champs qui ne correspondent pas à une association (where e1.f1 = e2.f2)

Exemple

- ❑ Si on veut tous les employés d'un département, la requête suivante n'est pas permise par la spécification JPA (bien que TopLink l'autorise) :

```
select d.employes
from Departement d
where d.nom = 'Direction'
```
- ❑ Une jointure est nécessaire :

```
select e
from Departement d join d.employes e
where d.nom = 'Direction'
```

Autres exemples

- ❑ `select e.nom
from Departement d
join d.employes e
where d.nom = 'Direction'`
- ❑ `select e.nom, parts.projet.nom
from Employe e
join e.participations parts`
- ❑ `select e.nom, d.nom
from Employe e, Departement d
where d = e.departement`
- ❑ `select e, p
from Employe e
join e.participations parts
join parts.projet p`

Jointure externe

- ❑ `select e, d
from Employe e left join e.departement d`
ramènera aussi les employés qui ne sont pas associés à un département

join fetch

- ❑ Permet d'éviter le problème des « N + 1 selects »
- ❑ L'entité placée à droite de **join fetch** sera créée en mémoire en même temps que l'entité de la clause select
- ❑ Le select SQL généré sera une jointure externe qui récupérera les données de toutes les entités associées en même temps que les données des entités principales de la requête

Exemple

- ❑ `select e
from Employe e join fetch e.departement`
- ❑ Cette requête récupérera tous les employés mais, en plus, l'appel de la méthode **getDepartement()** ne provoquera aucune interrogation de la base de données puisque le « **join fetch** » aura déjà chargé tous les départements des employés
- ❑ L'exemple suivant précharge les collections de participations aux projets

Exemple

```
String texteQuery =  
    "select e "  
    + " from Employe as e "  
    + "      join fetch e.participations";  
Query query = em.createQuery(texteQuery);  
listeEmployes =  
    (List<Employe>)query.getResultList();
```

Doublons possibles avec join fetch

- ❑ La requête SQL lancée par un join fetch fait une jointure pour récupérer les entités préchargées
- ❑ Ensuite, les entités préchargées sont enlevées des lignes du résultat pour qu'elles n'apparaissent pas dans le résultat du query
- ❑ Ce traitement, imposé par la spécification de JPA, peut occasionner des doublons dans le résultat si le select renvoie des valeurs
- ❑ Pour les éviter, il faut ajouter l'opérateur **DISTINCT** dans le texte de la requête ou placer le résultat dans une collection de type **Set**

- ❑ Il existe aussi des variantes « **outer join** » de **join fetch** pour récupérer dans le select des entités non jointes à une autre entité

Produit cartésien (1)

- ❑ Le préchargement par join fetch de plusieurs collections d'une même entité peut occasionner un phénomène nuisible aux performances
- ❑ En effet, le select SQL généré par le fournisseur de persistance peut récupérer un produit cartésien des éléments des collections
- ❑ Le fournisseur s'arrange pour ne garder que les informations nécessaires mais le select peut renvoyer un très grand nombre de lignes qui vont transiter par le réseau

Produit cartésien (2)

- ❑ Dans le cas où les collections contiennent de nombreux éléments il faut donc vérifier avec les logs du fournisseur si le select généré renvoie effectivement un trop grand nombre de lignes et changer de stratégie de récupération si c'est le cas (récupérer séparément les informations sur les collections)
- ❑ En effet, si 2 collections contiennent 20 éléments, et si 1000 entités principales sont renvoyées le select renverra 400.000 lignes ! (au lieu de 40.000 si on ramène les informations avec 2 join fetch séparés)

Exemple

- ❑ Une entité principale ep avec 2 associations de l'entité principale avec d'autres entités e2 et e3
- ❑ Pour récupérer en mémoire les entités principales et associées, le select généré risque d'être du type suivant (consulter les logs du fournisseur de persistance) :

```
select ep.*, e2.*, e3.*
from EP ep
  left outer join E2 e2 on ep.id_e2 =
e2.id
  left outer join E3 e3 on ep.id_e3 =
e3.id
```

Fonctions

- ❑ Fonctions de chaînes de caractères : **concat**, **substring**, **trim**, **lower**, **upper**, **length**, **locate** (localiser une sous-chaîne dans une autre)
- ❑ Fonctions arithmétiques : **abs**, **sqrt**, **mod**, **size** (d'une collection)
- ❑ Fonctions de date : **current_date**, **current_time**, **current_timestamp**
- ❑ Fonctions de regroupement : **count**, **max**, **min**, **avg**
- ❑ Spécification JPA pour plus d'informations,

Travail avec les collections

- ❑ Une expression chemin d'un select peut désigner une collection
- ❑ Exemples : **departement.employes**, **facture.lignes**
- ❑ La fonction **size** donne la taille de la collection
- ❑ La condition « **is [not] empty** » est vraie si la collection est [n'est pas] vide
- ❑ La condition « **[not] member of** » indique si une entité appartient à une collection

Exemples

- ❑ `select d`
`from Departement`
`where e.employees is empty`
- ❑ `select e`
`from Employe e`
`where :projet member of`
`e.participations.projet`

Parcours d'une collection

- ❑ La requête suivante ne donnera pas tous les produits liés à une facture ; elle provoquera une exception à l'exécution
~~`select distinct f.lignes.produit`~~
~~`from Facture as f`~~
- ❑ En effet, il est interdit de composer une expression de chemin en partant d'une expression qui désigne une collection (**f.lignes**)

Parcours d'une collection

- ❑ Le plus simple est d'utiliser un **join** dans le **from** avec un alias (« l » dans l'exemple ci-dessous) pour désigner un élément qui parcourt la collection :
`select distinct l.produit`
`from Facture as f join f.lignes as l`
- ❑ L'autre solution, « in », une ancienne syntaxe héritée d'EJB 2, n'est pas recommandée

Pagination du résultat

- ❑ **Query setMaxResults(int n)** : indique le nombre maximum de résultats à retrouver
- ❑ **Query setFirstResult(int n)** : indique la position du 1^{er} résultat à retrouver (numéroté à partir de 0)

Enchaînement des méthodes

- ❑ Les méthodes **setParameter**, **setMaxResults** renvoient le **Query** modifié
- ❑ On peut donc les enchaîner
- ❑ Exemple :
`em.createQuery(texteQuery)`
`.setParameter(nomParam, valeurParam)`
`.setFirstResult(30 * i + 1)`
`.setMaxResults(30)`
`.getResultList();`

Opérations de modification en volume

Utilité

- ❑ Pour les performances il est parfois mauvais de charger toutes les données à modifier dans des instances d'entités
- ❑ En ce cas, EJBQL permet de modifier les données de la base directement, sans créer les entités correspondantes

Cas d'utilisation

- ❑ Si on veut augmenter de 5% les 1000 employés de l'entreprise il serait mauvais de récupérer dans 1000 instances les données de chaque employés, de modifier le salaire de chacun, puis de sauvegarder les données
- ❑ Un simple ordre SQL

```
update employe
set salaire = salaire * 1.05
```

sera énormément plus performant

Exemple

```
em.getTransaction().begin();
String ordre =
    "update Employe e " +
    " set e.salaire = e.salaire * 1.05";
Query q = em.createQuery(ordre);
int nbEntitesModif = q.executeUpdate();
em.getTransaction().commit();
```

Syntaxe

- ❑ Les ordres de modification en volume référencent les classes et les propriétés des classes mais ils ne créent aucune entité en mémoire et ils ne mettent pas à jour les entités déjà présentes en mémoire
- ❑

```
update Entite as alias
set alias.prop1 = val1, alias.prop2 = val2,...
where condition
```
- ❑ La condition peut être aussi complexe que la condition d'un select

Remarques

- ❑ Les modifications doivent être faites dans une transaction
- ❑ Le plus souvent il faut isoler le lancement de ces opérations dans une transaction à part, ou au moins exécuter ces opérations au début d'une transaction avant la récupération dans la base d'entités touchées par l'opération
- ❑ En effet, les entités en mémoire ne sont pas modifiées par l'opération et elles ne correspondront alors donc plus aux nouvelles valeurs modifiées dans la base de données

Exceptions

Exceptions non contrôlées

- ❑ JPA n'utilise que des exceptions non contrôlées (descendantes de **RuntimeException**)
- ❑ JPA utilise les 2 exceptions **IllegalArgumentException** et **IllegalStateException** du paquetage **java.lang**
- ❑ Sinon, toutes les autres exceptions sont dans le paquetage **javax.persistence** et héritent de **PersistenceException**

Types d'exception

- ❑ **NonUniqueResultException**
- ❑ **NoResultException**
- ❑ **EntityNotFoundException**
- ❑ **EntityExistsException**
- ❑ **TransactionRequiredException**
- ❑ **RollbackException**
- ❑ **OptimisticLockException**

Exception et rollback

- ❑ Toute les exceptions de type **PersistenceException** provoquent un rollback, sauf **NonUniqueResultException** et **NoResultException**
- ❑ Rappel : si l'accès se fait par propriétés, une **RuntimeException** levée par une méthode d'accès à une propriété provoque un rollback de la transaction en cours

Exceptions dans les méthodes d'accès aux propriétés

- ❑ Les **RuntimeException** provoquent un rollback de la transaction en cours
- ❑ Les exception contrôlées sont enveloppées dans une **PersistenceException** (non contrôlée) et provoquent un rollback

Transaction

2 types de transactions

- ❑ Les transactions locales à une ressource, fournies par JDBC sont attachées à une seule base de données
- ❑ Les transactions JTA, ont plus de fonctionnalités que les transactions JDBC ; en particulier elles peuvent travailler avec plusieurs bases de données

Transactions dans Java EE

- ❑ Elles ne sont pas étudiées dans ce cours

Transactions dans Java SE (sans serveur d'applications)

- ❑ D'après la spécification JPA, dans Java SE, les fournisseurs de persistance doivent supporter les transactions locales à une ressource, mais ne sont pas obligés de supporter les transactions JTA
- ❑ La démarcation des transactions est choisie par le développeur
- ❑ Les contextes de persistance peuvent couvrir plusieurs transactions

EntityTransaction

- ❑ En dehors d'un serveur d'applications, une application doit utiliser l'interface **javax.persistence.EntityTransaction** pour travailler avec des transactions locales à une ressource
- ❑ Une instance de **EntityTransaction** peut s'obtenir par la méthode **getTransaction()** de **EntityManager**

EntityTransaction

```
public interface EntityTransaction {  
    public void begin();  
    public void commit();  
    public void rollback();  
    public void setRollbackOnly();  
    public boolean getRollbackOnly();  
    public void isActive();  
}
```

Exemple

```
EntityManager em;  
...  
try {  
    em.getTransaction().begin()  
    ...  
    em.getTransaction().commit();  
}  
finally {  
    em.close();  
}
```

Rollback (1)

- ❑ En cas de *rollback*,
 - *rollback* dans la base de données
 - le contexte de persistance est vidé ; toutes les entités deviennent détachées

Rollback (2)

- ❑ Les instances d'entités Java gardent les valeurs qu'elles avaient au moment du *rollback*
- ❑ Mais ces valeurs sont le plus souvent fausses
- ❑ Il est donc rare d'utiliser ces entités en les rattachant par un **merge** à un GE
- ❑ Le plus souvent il faut relancer des requêtes pour récupérer des entités avec des valeurs correctes

Transaction et contexte de persistance

- ❑ Quand un GE n'est pas géré par un container (c'est le cas en dehors d'un serveur d'applications) le contexte de persistance n'est pas fermé à la fin d'une transaction
- ❑ Quand un GE est géré par un container (c'est le cas avec un serveur d'applications) et que le contexte de persistance n'est pas de type « étendu » (pas étudié dans ce cours), le contexte est fermé à la fin d'une transaction

Synchronisation d'un GE avec une transaction

- ❑ Synchronisation d'un GE avec une transaction : le GE est enregistré auprès de la transaction ; un commit de la transaction provoquera alors automatiquement un *flush* du GE (le GE est averti lors du commit)
- ❑ En dehors d'un serveur d'applications (avec Java SE), un GE est obligatoirement synchronisé avec les transactions (qu'il a lancées par la méthode **begin()** de **EntityManager**)

Modifications et commit

- ❑ Les modifications effectuées sur les entités gérées sont enregistrées dans la base de données au moment d'un flush du contexte de persistance
- ❑ Si le GE est synchronisé à la transaction en cours, le commit de la transaction enregistre donc les modifications dans la base
- ❑ Les modifications sont enregistrées dans la base, même si elles ont été effectuées avant le début de la transaction (avant **tx.begin()** dans Java SE)

Erreurs et problèmes divers

- ❑ Démarrer une nouvelle transaction alors qu'il y en a déjà une en cours lancera une **IllegalStateException**
- ❑ **commit()** et **rollback()** lancent la même exception si aucune transaction n'est active
- ❑ Si une erreur survient lors de **rollback()**, une **PersistenceException** sera lancée
- ❑ Si une erreur survient lors de **commit()**, une **RollbackException** est lancée

Marquage pour un rollback (1)

- ❑ Si une erreur survient lors d'une opération de persistance (insertion, suppression, mise à jour) la transaction en cours est marquée comme devant être invalidée (rollback)
- ❑ Le lancement explicite de **rollback()** est à la charge du code de l'application (pas automatique)
- ❑ Si le code lance tout de même un commit à la place d'un rollback, une **RollbackException** est lancée

Marquage pour un rollback (2)

- ❑ Le code de l'application peut se prémunir contre ce problème en lançant la méthode **getRollbackOnly()** qui renvoie **true** si la transaction est marquée pour un rollback
- ❑ Le code de l'application peut lui-même marquer la transaction en cours pour un rollback en utilisant la méthode **setRollbackOnly()**
- ❑ Il est impossible d'enlever une marque pour un rollback

Concurrence

GE et threads

- ❑ Une fabrique de GE peut être utilisée sans problème par plusieurs threads
- ❑ Mais un GE ne doit être utilisé concurremment que par un seul thread

Entités et threads

- ❑ Les entités ne sont pas prévues pour être utilisées par plusieurs threads en même temps
- ❑ Si ça doit être le cas, l'application doit prendre toutes ses précautions pour éviter les problèmes
- ❑ C'est aussi le rôle de l'application d'empêcher qu'une entité ne soit gérée par plusieurs GE en même temps
- ❑ Le rôle du fournisseur de persistance n'intervient pas pour ces cas

Concurrence « BD »

- ❑ Le fournisseur de persistance peut apporter automatiquement une aide pour éviter les problèmes d'accès concurrents aux données de la BD, pour les entités gérées par un GE

Exemple de problème de concurrence « BD »

- ❑ Une entité est récupérée depuis la BD (par un **find** par exemple)
- ❑ L'entité est ensuite modifiée par l'application puis la transaction souhaite valider cette modification
- ❑ Si une autre transaction a modifié entre-temps les données de la BD correspondant à l'entité, la transaction ne doit pas être validée pour éviter les problèmes de mises à jour perdues

Gestion de la concurrence

- ❑ Par défaut, le fournisseur de persistance gère les problèmes d'accès concurrents aux entités gérées par un GE avec une stratégie optimiste
- ❑ JPA 1.0 ne spécifie rien pour les blocages pessimistes
- ❑ JPA 1.0 suppose aussi que la BD a le mode d'isolation « READ COMMITED » (voir cours sur les BD) et que les écritures dans la base n'ont lieu qu'au moment d'un *flush*

@Version

- ❑ Annote un attribut dont la valeur représente un numéro de version (incrémenté à chaque modification), utilisé pour savoir si l'état d'une entité a été modifié entre 2 moments différents
- ❑ L'attribut doit être de type **int**, **Integer**, **short**, **Short**, **long**, **Long** ou **java.sql.Timestamp** (si possible, éviter ce dernier type) et ne doit pas être **null** dans la base de données
- ❑ L'application ne doit jamais modifier un tel attribut (seul JPA le modifie)

Exemple

```
@Version  
private int version;
```

Entité « versionnée »

- ❑ C'est une entité qui possède un attribut qui possède l'annotation **@Version** (ou le tag correspondant dans les fichiers XML)

Protection des entités « versionnées »

- ❑ Les données de la BD correspondant à une entité versionnée ne peuvent être modifiées concurrentement par 2 transactions gérées par JPA : si une transaction t1 récupère les données, elle ne pourra valider des modifications sur ces données que si une autre transaction ne les a pas modifiées entre le moment où t1 les a lues et le moment où elle lance un commit

Protection des entités « non versionnées »

- ❑ Le fournisseur de persistance peut aussi effectuer automatiquement ce contrôle sur une entité non versionnée (par exemple en comparant les données lues avec les données de la base au moment du commit), mais JPA ne l'impose pas
- ❑ Pour la portabilité il vaut donc mieux versionner les entités sujettes à des accès concurrents

lock(entite, mode)

- ❑ Sert à protéger une entité contre les accès concurrents pour les cas où la protection offerte par défaut ne suffit pas
- ❑ Cette entité doit être versionnée (sinon, le traitement n'est pas portable) et déjà gérée

Modes de blocage

- ❑ L'énumération **LockModeType** (paquetage **javax.persistence**) définit 2 modes
- ❑ **READ** : empêche les lectures non répétables : les autres transactions peuvent lire l'entité mais ne peuvent pas la modifier
- ❑ **WRITE** : comme **READ**, mais en plus l'attribut de version est incrémenté, même si l'entité n'a pas été modifiée (utilité expliquée dans les transparents suivants) par la transaction dans laquelle le blocage a été effectué

Blocage READ

- ❑ Ce blocage est utile pour faire des bilans pour lesquels les données ne doivent pas être modifiées du début à la fin du traitement
- ❑ Avec ce blocage, si une transaction lit des données et si une autre transaction modifie ces données, alors les transactions ne peuvent être validées toutes les deux (celle qui échoue n'est pas spécifiée)
- ❑ La différence avec le mode de protection par défaut est que le problème arrivera, même si l'entité n'est pas modifiée par la transaction qui a bloqué l'entité

Blocage READ

- ❑ Le « blocage » réel peut n'être effectué qu'au moment du flush ou du commit, quelle que soit l'emplacement du **read** dans le code Java
- ❑ S'il y a un problème, une exception **OptimisticLockException** peut être lancée
- ❑ Le fournisseur de persistance peut choisir une stratégie pessimiste (du type « select for update ») ou optimiste pour implémenter le blocage

Une situation à éviter

- ❑ Un bilan qui calcule le total des salaires des employés pour chaque département doit interdire qu'un employé ne change de département pendant le traitement, en passant du département 10 au département 20
- ❑ Sinon, cet employé sera compté 2 fois : après avoir fait le calcul du total du département 10, on va faire une requête pour récupérer les employés du département 20 et retrouver l'employé déjà décompté dans le département 10

Utilité des blocages READ

- ❑ Pour éviter ce problème, les employés peuvent être bloqués en mode READ, au fur et à mesure de leur prise en compte pour le calcul du total des salaires de leur département
- ❑ Le blocage peut être pessimiste (select for update sur les lignes bloquées) ou optimiste ; dans ce dernier cas le traitement recevra une exception si l'optimisme était injustifié

WRITE

- ❑ Force une incrémentation du numéro de version
- ❑ Pour certains traitements il est intéressant de faire considérer que l'entité a été modifiée même si aucun de ses attributs n'a été modifié
- ❑ Dans l'exemple qui suit le blocage WRITE évite en quelque sorte les problèmes de lignes fantômes avec les associations qui concernent une entité

WRITE

- ❑ En effet, sans le blocage « WRITE », le fournisseur de persistance incrémente automatiquement le numéro de version si une entité est modifiée mais pas si les liens des associations qui partent de l'entité sont modifiées (voir cependant la remarque qui suit l'exemple donné dans les transparents suivants)

Utilité des blocages WRITE (1)

- ❑ Une entête de facture contient un champ dans lequel est stocké le total de la facture
- ❑ Supposons que le calcul de ce total est effectué par un processus à part du processus de l'ajout des lignes de la facture
- ❑ Lorsqu'une ligne de facture est ajoutée à une facture, l'en-tête de la facture n'est pas modifiée donc le fournisseur de persistance n'incrémente pas le numéro de version de l'en-tête

Utilité des blocages WRITE (2)

- ❑ Une ligne peut être ajoutée à une facture sans que le traitement qui calcule le total ne s'en rende compte ; il y aura inconsistance entre ce total et les lignes de la facture
- ❑ Si l'en-tête est bloquée en écriture pendant l'ajout d'une ligne de facture, le traitement qui effectue le calcul du total s'apercevra que la facture a changé pendant le calcul puisque le numéro de version aura été incrémenté et il n'y aura pas inconsistance

Remarque

- ❑ La spécification JPA ne dit pas avec précision quand le numéro de version est incrémenté
- ❑ Certains fournisseurs de persistance comme Hibernate incrémentent le numéro de version quand les collections d'une entité sont modifiées
- ❑ Dans ce cas le blocage Write sera tout de même utile si, par exemple, un des éléments de la collection est modifié, par exemple le prix d'un produit compris dans la facture

Entité détachée

- ❑ Les entités détachées sont surtout utiles dans les applications multi-tiers (avec Java EE et un serveur d'applications)
- ❑ Cette section survole les possibilités offertes par les entités détachées
- ❑ Les subtilités des entités détachées sont exposées dans le support complémentaire « JPA et serveurs d'applications »

Cas d'utilisation

- ❑ Une application multi-tiers
- ❑ Le serveur d'application récupère des données dans la BD
- ❑ Ces données sont passées à la couche client, montrées à l'utilisateur qui peut les modifier
- ❑ Les modifications sont repassées au serveur et enregistrées dans la BD
- ❑ Les entités détachées facilitent l'implémentation d'un tel cas d'utilisation

Utilisation des entités détachées

- ❑ Une fois détachées les entités peuvent être passées à la couche cliente
- ❑ La couche cliente peut modifier les entités détachées
- ❑ Ces entités peuvent ensuite être rattachées à un GE et les modifications effectuées dans la couche cliente peuvent être alors enregistrées dans la base de données lors d'un flush

Rattachement

- ❑ La méthode **merge** de **EntityManager** permet d'obtenir une entité gérée à partir d'une entité détachée
- ❑ Sa signature :
`<T> T merge(T entité)`
- ❑ Attention, l'entité passée en paramètre n'est pas rattachée ; c'est l'entité renvoyée par la méthode **merge** qui est rattachée ; cette entité a le même état et la même clé primaire que l'entité passée en paramètre

- ❑ Si l'entité passée en paramètre de merge est déjà gérée par le GE, elle est renvoyée par la méthode merge

État d'une entité détachée

- ❑ L'état d'une entité détachée peut ne pas être entièrement disponible

État d'une entité

- ❑ Pour des raisons de performances, l'état d'une entité gérée par un GE peut ne avoir été complètement récupéré dans la BD (récupération retardé ; *lazy*)
- ❑ Le reste de l'état ne sera récupéré, avec l'aide du GE, que lorsque l'entité en aura vraiment besoin
- ❑ Si l'entité est détachée alors qu'une partie de son état n'a pas encore été récupérée, la partie manquante de l'entité détachée ne sera pas disponible

Attribut disponible

- ❑ Un attribut persistant d'une entité est immédiatement disponible dans les 2 cas suivants :
 - l'attribut a déjà été utilisé
 - l'attribut n'a pas été marqué par **fetch=LAZY** (par défaut, les valeurs des attributs sont chargées en mémoire)

Association d'une entité détachée

- ❑ Si une association d'un objet détaché a le mode de récupération LAZY, il est possible que l'association ne puisse être récupérée (dépend des circonstances et des fournisseurs de JPA)

Association « lazy » disponible

- ❑ Si une association d'un objet détaché a le mode de récupération LAZY, il est possible de naviguer à travers cette association si
 - l'application a déjà effectué cette navigation
 - ou si l'association a été chargée par un **join fetch** lors d'une requête

Rendre accessible l'état d'une entité détachée

- ❑ 2 solutions pour accéder à tout l'état d'une entité détachée
- ❑ L'application récupère tout l'état de l'entité gérée avant le détachement
- ❑ L'application rattache l'entité par la méthode **merge** pour récupérer l'état manquant

Récupérer une association avant le détachement

- ❑ Si le but est une entité (OneToOne ou ManyToOne), il suffit de lire une des propriétés de l'entité
- ❑ Si le but est une collection (OneToMany ou ManyToMany), l'appel de la méthode **size()** de la collection suffit le plus souvent

Entité détachée et concurrence

- ❑ Avant que les modifications sur l'entité détachée ne soient enregistrées dans la BD, l'entité doit être rattachée à un contexte de persistance (celui d'origine ou un autre)
- ❑ A ce moment, ou au moment du commit qui enregistre vraiment les modifications, le contexte de persistance vérifie qu'il n'y a pas de conflit de concurrence

Conflit de concurrence

- ❑ Si les données de la BD associée à un objet détaché ont été modifiées depuis le détachement de l'objet, **merge** lance une exception, ou le commit échouera
- ❑ Tout se passe donc comme si un conflit de concurrence avait été détecté (avec une stratégie optimiste)

Détachement automatique

- ❑ Dans Java SE, quelques situations provoquent un détachement automatique des entités gérées :
 - Après un rollback
 - Après un clear du GE
 - Après la fermeture du GE

Configuration d'une unité de persistance

Fichier persistence.xml

- ❑ Les informations doivent être données dans un fichier **persistence.xml**
- ❑ Dans un environnement non géré, ce fichier doit se situer dans le répertoire META-INF d'un des répertoires du *classpath*
- ❑ Dans un serveur d'applications, le répertoire META-INF peut se trouver à divers endroits : dans WEB-INF/classes, un fichier jar de WEB-INF/lib, un fichier jar de la racine d'un fichier EAR,... (voir spécification pour la liste complète)

Configuration d'une unité de persistance

- ❑ Dans un environnement non géré par un serveur d'applications, il est nécessaire de donner les informations pour la connexion à la base de données
- ❑ Souvent une application n'utilise qu'un seul compte utilisateur dans la base de données ; en ce cas, le nom et le mot de passe de ce compte sont donnés dans le fichier persistence.xml

Configuration d'une unité de persistance

- ❑ Le fichier **persistence.xml** donne d'autres informations, comme les noms des classes entités, la configuration de l'environnement de mis au point (*logging*, affichage des ordres SQL lancés dans le SGBD,...) ou des propriétés particulières au driver du fournisseur de persistance

persistence.xml

```
<persistence
  xmlns="http://java.sun.com/xml/ns/persistence"
  version="1.0" >
  <persistence-unit name="Employes"
    transaction-type="RESOURCE_LOCAL">
    <class>p1.Employe</class>
    <class>p1.Dept</class>
    <properties>
      . . . <!-- Voir transparent suivant-->
    </properties>
  </persistence-unit>
</persistence>
```

Section properties

- ❑ La section **properties** dépend du fournisseur des fonctionnalités décrites dans la spécification JPA
- ❑ Elle contient les informations pour la connexion mais aussi d'autres informations pour le logging ou la création automatique des tables si elle n'existent pas déjà ; il faut consulter la documentation du fournisseur

Exemple de section properties

```
<properties>
  <property
    name="toplink.jdbc.driver"
    value="oracle.jdbc.OracleDriver"/>
  <property
    name="toplink.jdbc.url"
    value="jdbc:oracle:thin:@...:INFO "/>
  <property name="toplink.jdbc.user"
    value="toto"/>
  <property name="toplink.jdbc.password"
    value="mdp"/>
</properties>
```

Source de données

- ❑ Lorsque JPA est utilisé avec un serveur d'applications, une source de données fournie par le serveur d'applications est le plus souvent utilisée pour obtenir les connexions (voir **DataSource** dans le cours sur JDBC)
- ❑ On pourra alors trouver :

```
<persistence-unit name="Employe">
  <jta-data-source>jdbc/EmployeDS</jta-data-source>
</persistence-unit name=Employe>
```

Configuration dynamique (1)

- ❑ Les informations pour la connexion à la base, en particulier le nom et le mot de passe de connexion à la base, peuvent n'être connues qu'à l'exécution et pas au moment de l'écriture du code
- ❑ En ce cas, il est possible de ne donner les informations que lors de la création de la fabrique par la classe **Persistence**

Configuration dynamique (2)

- ❑ Il suffit de passer à la méthode **createEntityManagerFactory** une map qui contient les propriétés qui écrasent les valeurs contenues dans **persistence.xml**

Exemple de configuration dynamique

```
// Saisie du nom et du mot de passe
...
// Configuration de la connexion
Map props = new HashMap();
props.put("toplink.jdbc.user", nom);
props.put("toplink.jdbc.password", mdp);
EntityManagerFactory emf =
    Persistence
        .createEntityManagerFactory(
            "employees", props);
```

Les ajouts de TopLink

- ❑ TopLink a des extensions par rapport à la norme JPA
- ❑ Par exemple,
 - *logging*
 - génération automatique de tables
 - ...

Génération automatique des tables

- ❑ La propriété **toplink.ddl-generation** permet de créer automatiquement les tables au moment de la création de la fabrique de gestionnaires d'entités
- ❑ Cette propriété peut avoir 3 valeurs :
 - **none** : aucune création automatique
 - **create-tables** : les tables sont créées si elles n'existent pas déjà
 - **drop-and-create-tables** : si une table existe déjà, elle est d'abord supprimée avant d'être recréée (utile pour les tests)

Exemple

```
<properties>
...
<property name="toplink.ddl-generation"
    value="drop-and-create-tables"/>
</properties>
```

- ❑ **toplink.ddl-generation.output-mode** indique ce qui sera fait avec les fichiers DDL
- ❑ Valeurs possibles :
 - **sql-script** génère les fichiers mais ne les exécute pas
 - **database** exécute les ordres DDL mais ne génère pas les fichiers
 - **both** génère les fichiers et les exécute (valeur par défaut)

- ❑ Propriété **toplink.application-location** indique le nom du répertoire qui contiendra les fichiers contenant les ordres DDL de création et de suppression des tables (le répertoire courant par défaut)

- ❑ **toplink.create-ddl-jdbc-file-name** indique le nom du fichier qui contiendra les ordres de création des tables ; par défaut, **createDDL.jdbc**
- ❑ **toplink.drop-ddl-jdbc-file-name** idem pour la suppression des tables ; par défaut, **dropDDL.jdbc**

logging (1)

- ❑ La propriété **toplink.logging.level** peut avoir les valeurs
 - **OFF** : aucune information
 - **SEVERE** : uniquement les erreurs
 - **WARNING** : les avertissements (et les erreurs)
 - **INFO** (valeur par défaut) : assez peu d'information en plus
 - **CONFIG** : donne des informations au moment du déploiement sur la configuration

logging (2)

- **FINE** : donne des informations sur les ordres SQL utile pendant les tests et la mise au point
- **FINER** : encore plus d'informations, par exemple sur les transactions
- **FINEST** : encore plus d'informations, par exemple sur l'utilisation des séquences

Fichiers XML

Placement des fichiers XML

- ❑ Par défaut les fichiers XML contenant les méta données sont placées dans le fichier **META-INF/orm.xml**, sous un répertoire du *classpath*
- ❑ Il est possible d'indiquer d'autres emplacements avec le tag **<mapping-file>** dans le fichier **persistence.xml** qui définit l'unité de persistance

Exemple

```
<persistence-unit name="xxxx">
  ...
  <mapping-file>META-INF/queries.xml
</mapping-file>
  <mapping-file>META-INF/entities.xml
</mapping-file>
</persistence-unit>
```

Méta-données par défaut

- ❑ L'élément **<persistence-unit-defaults>** contient des valeurs par défaut qui s'appliquent à toutes les entités de l'unité de persistance
- ❑ Les sous-éléments peuvent être : **<schema>** (donner le nom du schéma relationnel), **<catalog>** (idem **<schema>** pour les SGBD qui supportent les catalogues), **<access>** (accès pour toutes les classes non annotées), **<cascade-persist>** (pour imposer la persistance par transitivité) et **<entity-listeners>** (écouteurs par défaut)

Fichiers XML – annotations

- ❑ Avantages des annotations :
 - méta données proches du code
 - pas besoin de donner le contexte comme dans les fichiers XML
 - plus simple
- ❑ Inconvénients :
 - changement des méta données nécessite une recompilation
- ❑ Les informations données dans les fichiers XML l'emportent sur les annotations

Fichiers XML – annotations

- ❑ Les annotations sont le plus souvent utilisées
- ❑ Les fichiers de configuration XML peuvent être préférés lorsque l'information
 - est dépendante de l'environnement d'exécution ; par exemple pour les noms de tables ou de colonnes
 - concerne plusieurs classes ; par exemple **@TableGenerator** qui donne le nom de la table qui génère automatiquement les clés d'identification des entités

Exemple de fichier orm.xml (en-tête)

```
<?xml version="1.0" encoding="UTF-8" ?>
<entity-mappings
xmlns="http://java.sun.com/xml/ns/persistence/orm"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation =
"http://java.sun.com/xml/ns/persistence/orm_1_0.xsd" version="1.0">
```

Exemple de fichier orm.xml (fin)

```
<entity class="jpa.Employe">
  <table name="EMP" />
  <named-query name="findEmpByName"
    <query>select e from Employe e where
e.nom like :nomEmploye</query>
  </named-query>
</entity>
</entity-mappings>
```

JPA et DAO

Utilité des DAO ?

- ❑ La possibilité de détacher et rattacher une entité rend souvent les DTO inutiles (voir cours sur DAO et modèles de conception associés)
- ❑ Puisque JPA augmente fortement la portabilité du code chargé de la persistance, est-ce que les DAOs sont encore nécessaires ?

Les DAOs ne sont pas toujours utiles

- ❑ Pour les petites applications à existence courte, avec peu de fonctionnalités « métier », l'utilisation directe de JPA depuis le code métier (sans utilisation de DAOs) simplifie le code lié à la persistance
- ❑ Les DAO ne sont alors pas vraiment utiles

Les DAOs sont encore utiles

- ❑ Pour les applications plus complexes, les DAOs peuvent apporter une meilleure abstraction et une meilleure factorisation du code lié à la persistance
- ❑ Les détails techniques liés à JPA (ou à une version de JPA) peuvent être cachés dans la couche DAO
- ❑ Les DAOs pourront aussi faciliter le passage éventuel à un autre type de persistance, par exemple à un SGBD objet

Exemple de factorisation du code

- ❑ Un DAO peut contenir des méthodes spécialement adaptées au « métier »
- ❑ Par exemple, une méthode qui renvoie la liste des candidats qui ont obtenu leur examen avec une certaine mention, passée en paramètre
- ❑ Cette méthode peut être appelée de plusieurs endroits du code « métier »

Variantes des DAOs

- ❑ Parmi toutes les variantes des méthodes des DAOs (voir cours sur les DAOs), celles qui prennent en paramètre ou qui renvoient des objets (des entités JPA dans ce cas) sont le plus souvent choisies
- ❑ En effet, les entités détachées peuvent jouer le rôle des DTOs qui sont donc inutiles, et les variantes qui utilisent plusieurs paramètres dont les valeurs correspondent aux propriétés des objets compliquent inutilement le code

DAOs génériques

- ❑ La généricité permet d'écrire une classe mère générique de tous les DAOs, ce qui allège d'autant les classes filles représentant chacune des classes DAOs
- ❑ Le type des entités gérées et le type de l'identificateur sont les paramètres de type de ce DAO générique
- ❑ Les transparents suivants sont un exemple d'implémentation que vous pouvez modifier à votre convenance

Interface DAO générique

```
public interface DaoGenerique<T, ID extends
Serializable> {
    T findById(ID id);
    List<T> findAll();
    void create(T objet);
    void delete(T entite);
    T update(T entite);
}
```

Interface DAO pour une entité

- ❑

```
public interface StyloDao
    extends DaoGenerique<Stylo, Long> { }
```
- ❑ On peut ajouter des méthodes adaptées au « métier », comme cette méthode qui renvoie toutes les marques de stylos en vente :

```
public interface StyloDao
    extends DaoGenerique<Stylo, Long> {
    List<String> findMarques();
}
```

Classe DAO générique (1)

```
public abstract class
DaoGeneriqueJpa<T, ID extends Serializable>
implements DaoGenerique<T, ID> {
    private Class<T> classeEntite;
    private EntityManager em;
    public DaoGeneriqueJpa() {
        // classe concrète de T passée à find
        this.classeEntite = (Class<T>)
            ((ParameterizedType)getClass().
            .getGenericSuperclass())
            .getActualTypeArguments()[0];
    }
}
```

Classe DAO générique (2)

```
public void insert(T objet) {
    em.persist(objet);
}
public void delete(T objet) {
    em.remove(objet);
}
public T findById(ID id) {
    return em.find(classeEntite, id);
}
public T update(T entite) {
    return em.merge(entite);
}
```

Classe DAO générique (3)

```
public void
setEntityManager(EntityManager em) {
    this.em = em;
}
protected EntityManager
getEntityManager() {
    return this.em;
}
}
```

DAO pour une entité

```
public class StyloDaoJpa
    extends DaoGeneriqueJpa<Stylo, Long>
    implements StyloDao {
    public List<Stylo> findAll() {
        Query query = getEntityManager()
            .createNamedQuery("Stylo.findAll");
        return
            (List<Stylo>)query.getResultList();
    }
}
```

Cache des données

Utilité

- ❑ Tous les fournisseurs de persistance utilisent un cache de second niveau (en plus du contexte de persistance) pour éviter des accès aux bases de données, et donc améliorer les performances
- ❑ La spécification JPA ne donne aucune indication sur les caches de 2^{ème} niveau ; il faut consulter la documentation du fournisseur de persistance pour connaître les possibilités

Utilisation des caches

- ❑ Il n'est pas toujours facile d'utiliser correctement les caches de 2^{ème} niveau
- ❑ Le plus souvent il est possible d'indiquer quelles classes utiliseront ces caches, et sur quel mode
- ❑ Par exemple, il peut être très intéressant d'utiliser un cache « *read-only* » pour des classes dont les données ne changent que très rarement comme les noms des pays

Problèmes éventuels

- ❑ Parfois ces caches posent des problèmes si la base de données est utilisées en parallèle par d'autres applications
- ❑ Par exemple, si des entités liées à une entité sont supprimées de la base en dehors de l'application, le cache peut penser modifier une ligne avec un UPDATE au lieu d'en insérer une nouvelle avec un INSERT

Solutions (1)

- ❑ L'utilisation de la méthode **refresh** de **EntityManager** permet de récupérer dans la base de données des données « fraîches », sans passer par le cache
- ❑ Un blocage pessimiste peut parfois être la solution si on souhaite interdire la modification de lignes par une autre application (implémentation dépendante du fournisseur de persistance)

Solutions (2)

- ❑ Parfois la seule solution est de vider le cache ou d'indiquer au fournisseur de ne pas utiliser le cache pour effectuer certaines opérations (implémentation dépendante du fournisseur de persistance)

Optimisations

- ❑ Les performances d'une application peuvent être grandement améliorées par
 - un choix adapté du mode de récupération des entités associées
 - l'utilisation d'opérations de modifications en volume, sans création d'entités
 - une bonne utilisation du cache de 2^{ème} niveau

Méthode *callback* et *listener*

Méthodes « *callback* »

- ❑ Des méthodes peuvent être annotées pour indiquer qu'elles seront appelées par le fournisseur de persistance quand une entité passera dans une nouvelle étape de son cycle de vie
- ❑ Ces méthodes peuvent appartenir à une classe entité (**entity**) ou classe mère « **mappedSuperclass** » ou à une classe « écouteur » (*listener*)
- ❑ Une méthode peut être annotée par plusieurs de ces annotations

Annotations

- ❑ **@PrePersist** : quand persist (ou merge) s'est terminé avec succès
- ❑ **@PostPersist** : après l'insertion dans la BD
- ❑ **@PreRemove** : quand remove est appelé
- ❑ **@PostRemove** : après suppression dans la BD
- ❑ **@PreUpdate** : avant modification dans la BD
- ❑ **@PostUpdate** : après modification dans la BD
- ❑ **@PostLoad** : après la lecture des données de la BD pour construire une entité

Callback dans fichier XML

- ❑ Les méthodes callback peuvent aussi être indiquées dans un fichier XML
- ❑ Un fichier XML peut aussi indiquer des *listeners* par défaut qui seront appelées pour toutes les entités, dans le sous-élément **<entity-listeners>** de l'élément **<persistence-unit-defaults>**

Exemples d'utilisation

- ❑ Un trigger de la base de données peut donner les valeurs de l'utilisateur qui a modifié pour la dernière fois une entité, avec la date de cette modification
- ❑ Pour remplacer un tel trigger, il est possible d'ajouter une méthode annotée par **@PrePersist** qui remplit ces valeurs dans les entités
- ❑ Une méthode callback peut aussi initialiser des attributs non persistants d'une entité

Callback dans une entité

- ❑ Ces méthodes ne doivent aucune paramètre et le type retour doit être **void**
- ❑ Elles ne doivent pas avoir de clause **throws**
- ❑ Si elles renvoient une exception non contrôlée, les éventuelles méthodes callbacks suivantes ne sont pas appelées et la transaction est marquée pour un *rollback* (elle ne pourra pas être validée)
- ❑ Une seule méthode callback d'un certain type (par exemple **PrePersist**) par entité

Callback dans un écouteur

- ❑ La signature doit avoir un paramètre compatible avec le type de l'entité gérée pour qu'il puisse contenir l'instance de l'entité
- ❑ « Compatible » signifie que le type doit être un « surtype » de la classe entité : la classe de l'entité, ou une classe mère, ou une interface implémentée par la classe de l'entité
- ❑ Une classe *listener* ne doit contenir aucune variable d'instance (elle doit être « sans état ») et doit avoir un constructeur sans paramètre

Attacher un écouteur

- ❑ L'annotation **@EntityListeners** permet d'attacher un ou plusieurs écouteurs à une classe entité

- ❑ Exemple :

```
@Entity
@EntityListeners({C1.class, C2.class})
public class Entite {
```

Ordre d'appel

- ❑ Lorsque un événement du cycle de vie survient, les différentes méthodes sont appelées dans cet ordre :
 1. Méthodes des listeners par défaut
 2. Méthodes des listeners
 3. Méthode de l'entité en cause

Bibliographie

Sites Web (1)

- ❑ Spécification officielle de JPA :
<http://jcp.org/aboutJava/communityprocess/final/jsr220/index.html> (cliquer sur le 1^{er} bouton
« *download the Specification for evaluation or to build an application that uses the Specification* »)
- ❑ Toplink :
<http://www.oracle.com/technology/products/ias/toplink/jpa/index.html>

Sites Web (2)

- ❑ Hibernate :
http://www.hibernate.org/hib_docs/entitymanager/reference/en/html/
- ❑ Kodo :
http://e-docs.bea.com/kodo/docs40/full/html/ejb3_overview.html

Livres

- ❑ Pro EJB 3 – JPA
de Mike Keith et Merrick Schincariol
Edition Apress
(en anglais)
- ❑ Java Persistence with Hibernate
de Christian Bauer et Gavin King
Edition Manning
(en anglais)